TD n°4 – Héritage et Polymorphisme

1. Exemple de classes dérivées

Écrire ce qu'affiche ce programme dans la console après exécution de `TestInit` ?

```
public class A {
    // ici, exceptionnellement, pas d'encapsulation
    public int n;
    public int p=10;

    public A (int nn) {
        System.out.println("Entrée Constr. A : n="+n+" p="+p);
        n = nn;
        System.out.println("Sortie Constr. A : n="+n+" p="+p);
    }
}

public class B extends A {
    public int q=25;

    public B (int n, int pp) {
        super(n);
        System.out.println("Entrée Constr. B : n="+n+" p="+p+" q="+q);
        p = pp;
        q = 2*n;
        System.out.println("Sortie Constr. B : n="+n+" p="+p+" q="+q);
    }
}

public class TestInit {
    public static void main (String[] args) {
        A a = new A(5);
        A b = new B(5, 3);
    }
}
```

2. Formes géométriques

Dans un Java Project TD4, et à partir de la classe `Rectangle` disponible sur *Moodle*, écrire une classe `Carre` qui hérite de `Rectangle`. Cette nouvelle classe ne possède pas d'attribut spécifique.

- Écrire un constructeur pour la classe `Carre` qui prend en paramètre la longueur d'un côté du carré. Comment faire pour faire correspondre cette caractéristique avec les attributs de la classe `Rectangle` ?
- Comme pour les points colorés, redéfinir dans `Carre` les méthodes de la classe `Rectangle` qui ne s'appliquent pas directement aux carrés.
- Créer une méthode `main` et instancier des carrés et des rectangles afin de tester les constructeurs et méthodes.

3. Polymorphisme

Quel résultat fournit le programme suivant ? Expliquer.

```
class A { public void affiche() { System.out.print ("Je suis un A ") ; } }

class B extends A {}

class C extends A { public void affiche() { System.out.print ("Je suis un C ") ; } }

class D extends C { public void affiche() { System.out.print ("Je suis un D ") ; } }

class E extends B {}

class F extends C {}

public class Poly {
	public static void main (String arg[]) {
		A a = new A() ; a.affiche() ; System.out.println();

		B b = new B() ; a=b;
		b.affiche() ; a.affiche() ; System.out.println();

		C c = new C() ; a=c;
		c.affiche() ; a.affiche() ; System.out.println();

		D d = new D() ; a=d; c=d;
		d.affiche() ; a.affiche() ; c.affiche() ; System.out.println();

		E e = new E() ; a=e; b=e;
		e.affiche() ; a.affiche() ; b.affiche() ; System.out.println();

		F f = new F() ; a=f; c=f;
		f.affiche() ; a.affiche() ; c.affiche() ; System.out.println();
	}
}
```

4. Les points colorés

Avant de commencer la classe `ColoredPoint` , vous devez Compléter votre classe Point du TD1 en redéfinissant les méthodes *`equals`* et `toString` de la classe `Object`. Pour cela, vous devez faire vérifier au compilateur que vous êtes bien dans une redéfinition de méthode écrivant explicitement la balise `@Override` juste avant chaque méthode redéfinie. En particulier :

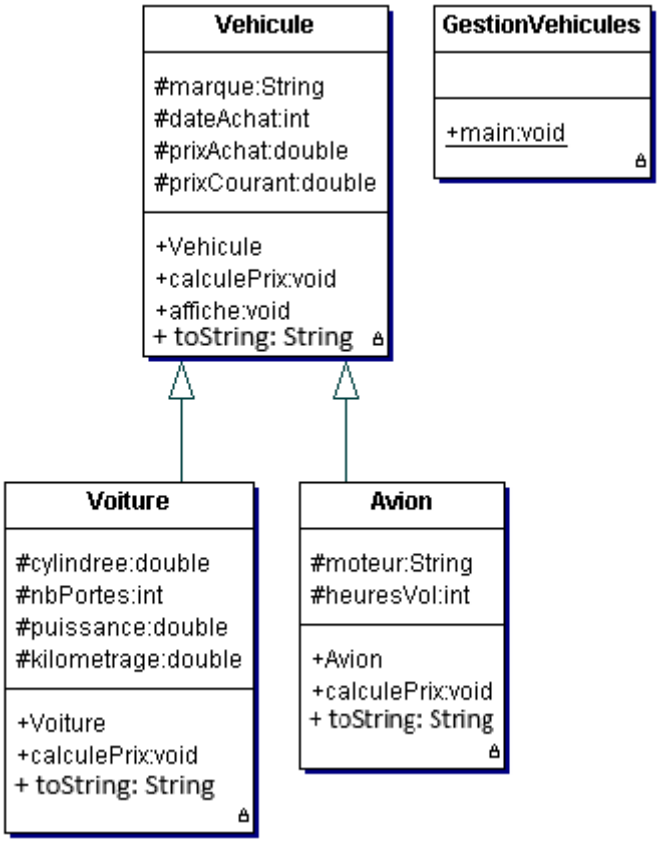
1) Ajouter la balise juste avant la méthode `toString` et vérifiez que vous n'avez pas d'erreur de compilation.
2) Ajouter ensuite une redéfinition de la méthode *`equals`* d'`Object` avec la balise `@Override` qui va vérifier si 2 objets `Point` ont les mêmes coordonnées ou pas.
3) Lisez bien la définition de la méthode `equals` dans la documentation de la classe `Object` afin de répondre aux exigences de cette méthode (en particulier si un des objets est `null`).

Créer maintenant la classe `ColoredPoint` vue en cours qui hérite de la classe `Point`. La compléter avec un accesseur de consultation de la couleur et une méthode `memeCouleur` qui retourne *vrai* si le point coloré courant et celui passé en paramètre sont de la même couleur, *faux* sinon.

- Créer une classe pour la méthode `main`. Instancier des points et des points colorés et tester les méthodes de la classe dérivée et de la classe de base.
- Dans `ColoredPoint` redéfinir :
  - La méthode *`equals`* de la classe `Object` afin de tester l'égalité entre 2 points colorés. Les coordonnées ainsi que la couleur doivent être identiques.
    → *Faire appel à equals de la classe `Point` et `memeCouleur` de la classe `ColoredPoint`*
  - la méthode `toString` de la classe `Object` afin d'afficher en plus des coordonnées, la couleur du point.
    → *Faire appel à toString() de la classe `Point`*
  - projX() → renvoie un `ColoredPoint` projeté de la même couleur
  - projY() → renvoie un `ColoredPoint` projeté de la même couleur
- Surcharger les méthodes de projection dans la classe `ColoredPoint` afin de préciser en paramètre une nouvelle couleur pour le point projeté.
- Dans le main, créer un tableau de points pouvant référencer à la fois des points « simples » et des points colorés.
  Ce tableau contiendra :
  - 5 points de coordonnées (0.0, 5.0), (1.0, 4.0), (2.0, 3.0), (3.0, 2.0), (4.0, 1.0) ;
  - 5 points colorés de coordonnées (1.0, 1.0), …, (5.0, 5.0) et de couleurs rouge, bleu, vert, noir, blanc (peu importe la correspondance coordonnées-couleur).

  1. Vous ferez appel aux différentes méthodes des classes `Point` et `ColoredPoint` sur ce tableau (`toString, getDistance, projX, projY, equals`, …).
  2. Pour chaque objet du tableau, tester les méthodes `instanceof` et `getClass().getName()` vues en cours, pour observer les effets du polymorphisme.
- Enfin, créer un nouvel objet de type `ColoredPoint` et affectez-lui le point bleu contenu dans votre tableau que vous devez rechercher (**on suppose que l'on ne connait pas la position du point bleu dans le tableau**). Quelle manipulation devez-vous faire et pourquoi ?

5. Flotte de véhicules



Reproduire l'arborescence d'héritage ci-dessus.

1. La méthode `affiche()` permet d'afficher l'état de l'instance, c'est-à-dire la valeur de ses attributs. Elle fera donc un *println* de l'objet, ce qui appellera de manière implicite la méthode `toString()` adéquate.
2. La méthode `calculePrix(int anneActuelle)` dans la classe `Vehicule` fixe le prix courant égal au prix d'achat moins 1% par année (entre la date d'achat et la date actuelle).
   - Redéfinissez cette méthode dans les deux classes dérivées `Voiture` et `Avion` de sorte à calculer le prix courant en fonction de certains critères, et mettre à jour l'attribut correspondant au prix courant.
     - Pour une voiture, le prix courant est égal au prix d'achat, moins :
       - 2% pour chaque année depuis l'achat jusqu'à la date actuelle
       - 5% pour chaque tranche de 10000 kms parcourus (on arrondit à la tranche la plus proche)
       - 10% s'il s'agit d'un véhicule de marque "Renault" ou "Fiat" (ou d'autres marques de votre choix)

       et **plus** 20% s'il s'agit d'un véhicule de marque "Ferrari" ou "Porsche" (idem).

     - Pour un avion, le prix courant est égal au prix d'achat, moins :
       - 10 % pour chaque tranche de 100 heures de vol s'il s'agit d'un avion à hélices.
       - 10 % pour chaque tranche de 1000 heures de vol pour les autres types de moteurs.

Le prix doit rester positif (donc s'il est négatif, le mettre à 0). De même, le prix actuel ne doit pas être plus important que le prix d'achat.

3. Exécuter la classe `GestionVehicules` donnée dans *Moodle*. Vérifier l'exactitude des résultats obtenus.
4. Améliorer la modélisation précédente en rendant *private* les attributs des différentes classes afin d'assurer une bonne encapsulation, et en ajoutant les méthodes *getters/setters* (accesseurs/mutateurs) afin de récupérer la valeur de chaque champ dans la classe correspondante.
5. Améliorer la méthode `main` en tenant compte du fait que tous les véhicules ont un type commun: `Vehicule`.

6. Bibliothèque

On veut modéliser la gestion d'une bibliothèque : on définira un certain nombre de classes : `Main`, `Ouvrage`, `Bibliotheque`, `Periodique`, `CD`, `Livre`.

- Les livres auront comme propriétés : auteur, titre, éditeur ;
- les périodiques: nom, numéro, périodicité ;
- les CDs: titre, auteur.

De plus tous les ouvrages auront une date d'emprunt (potentiellement nulle), une cote (le numéro par ordre de création). On implémentera également sur chaque objet une méthode `toString()` renvoyant toutes les informations sur l'ouvrage sous forme d'une chaîne de caractères.

La classe `Bibliotheque` permettra de stocker dans une structure les ouvrages (ajout et suppression, la suppression prenant en argument la cote de l'ouvrage). Elle aura également une méthode `toString()` affichant le nombres d'ouvrages, puis chaque ouvrage successivement.

La classe `Main` ne contiendra que la méthode main et testera la bibliothèque en y insérant et supprimant quelques ouvrages, puis en affichant le contenu de la bibliothèque.

1. Représentez les différentes classes dans un graphe d'héritage. On mettra en évidence pour chaque classe les méthodes et les champs qu'elle définit, redéfinit ou hérite. On souhaite que tous les champs soient déclarés privés et que l'on puisse y accéder de l'extérieur que par des méthodes.
2. Implémentez les classes ci-dessus. Pour la classe `Bibliotheque` on utilisera un tableau de longueur suffisante (on vérifiera quand même à chaque opération d'insertion que l'on ne dépasse pas les bornes du tableau).